

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Debugging String Matching Code (Inner Loop Starts with Wrong Text Offset in C) The following brute force string matching implementation in C compares the pattern using <code>text[j]</code> instead of <code>text[i + j]</code>, which ignores the current alignment and repeatedly checks only the beginning of the text. Carefully analyze why pattern matching must shift the comparison window for each outer-loop position. Apply systematic debugging so that the pattern is compared against the correct substring of the text at every alignment. Optimize the corrected implementation for clarity while preserving the brute force strategy. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments. <pre>#include <stdio.h> #include <string.h> void stringMatch(char text[], char pattern[]) { int n = strlen(text), m = strlen(pattern); int i, j; for (i = 0; i <= n - m; i++) { for (j = 0; j < m; j++) { if (text[j] != pattern[j]) // ERROR break; } if (j == m)</pre>	Q1

	<pre> printf("Match at %d\n", i); } }</pre>	
--	---	--

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I **M.Lakshmi Sahithi-192424280** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				

Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging Analysis

The error is in the inner loop:

```
if (text[j] != pattern[j]) // ERROR
```

Why is this wrong?

- The outer loop variable **i** represents the current starting position (alignment) in the text.
- For each alignment, the pattern should be compared with the substring starting at text[i].
- Using text[j] always compares the pattern with the beginning of the text, ignoring the current alignment.

For example:

Text = "ABCDEF"

Pattern = "CDE"

When **i = 2**, the comparison should be:

text[2] vs pattern[0] -> C vs C

text[3] vs pattern[1] -> D vs D

text[4] vs pattern[2] -> E vs E

But the incorrect code compares:

text[0] vs pattern[0] -> A vs C

text[1] vs pattern[1] -> B vs D

text[2] vs pattern[2] -> C vs E

Thus, the algorithm never checks the correct substring.

Correct Fix

Replace

`text[j]`

with

`text[i + j]`

This shifts the comparison window according to the current alignment.

Corrected C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Function to perform brute force string matching
```

```
void stringMatch(char text[], char pattern[])
```

```
{
```

```
    int n = strlen(text);
```

```
    int m = strlen(pattern);
```

```
    int i, j;
```

```

// Try every possible starting position
for (i = 0; i <= n - m; i++)
{
    // Compare pattern with current substring
    for (j = 0; j < m; j++)
    {
        if (text[i + j] != pattern[j])
            break;    // Mismatch found
    }

    // If all characters matched
    if (j == m)
    {
        printf("Pattern found at index %d\n", i);
    }
}

int main()
{
    char text[] = "ABABDABACDABABCABAB";
    char pattern[] = "ABABC";

    stringMatch(text, pattern);

    return 0;
}

```

}

Sample Output

Pattern found at index 10

Time Complexity Analysis

Let:

- **n** = length of the text
- **m** = length of the pattern

Outer Loop

Runs:

$$n - m + 1$$

times.

Inner Loop

In the worst case, compares all **m** characters for every alignment.

Worst-Case Time Complexity

[

$$O((n-m+1) \times m) \approx O(nm)$$

]

Best-Case Time Complexity

If the first character mismatches at every alignment:

[
O(n)
]

Space Complexity

Only a few integer variables are used.

[
O(1)
]

Summary

- **Bug:** Used `text[j]` instead of `text[i + j]`.
- **Reason:** The comparison window was not shifted with each outer-loop iteration.
- **Fix:** Replace `text[j]` with `text[i + j]`.
- **Correct Time Complexity:**
 - **Worst Case:** $O(nm)$
 - **Best Case:** $O(n)$
 - **Space Complexity:** $O(1)$

